R CODE FOR TEXT EXAMPLES

Regression Analysis: A Practical Introduction, 3rd Edition

Jeremy Arkes

There are many good resources for learning R. A few good ones are:

- To get started with the software … https://cran.cnr.berkeley.edu/
- An online book on regression analysis in R: https://bookdown.org/ccolonescu/RPoE4/
- Many useful lessons:
  - https://www.princeton.edu/~otorres/
  - https://dss.princeton.edu/training/RStata.pdf
  - https://www.statmethods.net/index.html
  - http://revolution-computing.typepad.com/.a/6a010534b1db25970b019b01594d79970c-pi
  - https://a-little-book-of-r-for-time-series.readthedocs.io/en/latest/src/timeseries.html#time-series-analysis for time series.

Note that there are some cases in which the R program gives slightly different results from the Stata program for the same data and regression. This is the case for various types of models, including weighted models and heteroskedasticity corrections.

Also note that I am not that experienced with R coding. I relied on the "Stata to R Code Translator" on chatgpt.com: https://chatgpt.com/g/g-dDlKTFvbT-stata-to-r-code-translator.

Importing comma-delimited files

```
income <- read.table("c:/data/book/text_files/income_data.csv",
header=TRUE, sep=",", row.names="id")
```

To see data, just submit the name of the data set.
income

To remove the data object, income:

```
rm(income)
```

The following are general R commands:

- setwd("c:/data/book/text_data/”)
  (Here, you indicate the directory on your computer in which the data are located.)
- using # at beginning of a line makes R not run the line, so this can be used for notes.

CHAPTER 2

Section 2.3.2

```
income = read.csv("income.csv", header = TRUE)
names(income)
#simple reg model [equation (2.4a & 2.4b)]
reg1 <- lm(income ~ educ, data=income)
# show results
summary(reg1)
```

Section 2.7

```
#to gauge relative performance [equation (2.17)];
baseball = read.csv("baseball_payroll_wins.csv", header = TRUE)
reg2 <- lm(wins ~ payroll, data=baseball)
summary(reg2)
```

Section 2.9

```
#multiple reg model [equation (2.21)];
income = read.csv("income.csv", header = TRUE)
reg3 <- lm(income ~ educ + afqt, data=income)
summary(reg3)

#using adjusted-educ instead [equation (2.22)];
reg4 <- lm(educ ~ afqt, data = income)
summary(reg4)
# Predict educhat (fitted values from model1)
income$educhat <- predict(model1)
# Create adjeduc by subtracting educhat from educ
income$adjeduc <- income$educ - income$educhat
# Second regression model: income ~ adjeduc
reg5 <- lm(income ~ adjeduc, data = income)
summary(reg5)
```

CHAPTER 3

Section 3.1

```
income = read.csv("income.csv", header = TRUE)
#using dummy variables [equation (3.1);
reg1 <- lm(income ~ educ + afqt + black + hisp, data=income)
```

Section 3.2.3

```
studyhours = read.csv("study_hours.csv", header = TRUE)

#Y = test score;
#H = hours studied;

studyhours$hours_sq <- studyhours$hours * studyhours$hours
studyhours$hours_gt8 <- as.numeric(studyhours$hours > 8)
studyhours$hours_beyond8 <- (studyhours$hours - 8) *
studyhours$hours_gt8
studyhours$hours_upto8 <- studyhours$hours * (1 -
studyhours$hours_gt8) + 8 * (studyhours$hours_gt8)

#equations 3.13 – 3.15;
reg6 <-lm(score ~ hours, data=studyhours)
summary(reg6)
studyhours$pred1 <- fitted(reg6)

reg7 <-lm(score ~ hours + hours_sq, data=studyhours)
summary(reg7)
studyhours$pred2 <- fitted(reg7)

reg8 <-lm(score ~ hours + hours_gt8 + hours_upto8 +
hours_beyond8, data=studyhours)
summary(reg8)
studyhours$pred3 <- fitted(reg8)
```

Section 3.3

```
MJ = read.csv("MJ-UR-new.csv", header = TRUE)
#Table 3.2;
reg9 <- lm(mj0910 ~ ur0910, data=MJ)
summary(reg9)
reg10 <- lm(mj0910 ~ ur0910, weights=(MJ$sqrtpop2010), data=MJ)
reg10 <- lm(mj0910 ~ ur0910, data=MJ, weights=sqrtpop2010)
```

```
summary(reg10)
reg11 <- lm(mj0910 ~ ur0910, data=MJ, weights=pop2010)
summary(reg11)
```

Section 3.4

```
#Section 3.4, Table 3.3
income = read.csv("income.csv", header = TRUE)
reg12 <- lm(income ~ educ + afqt + age + black + hisp,
data=income)
summary(reg12)
#Summarize the variables income, educ, afqt, age, black, hisp
summary(df[, c("income", "educ", "afqt", "age", "black",
"hisp")])
```

CHAPTER 4

Section 4.2;

```
*This creates a different randomized set of 100 observations
from the observations used in this section;

# Load the dataset
data = read.csv("income.csv", header = TRUE)
# Generate a random number only for observations with income <
150000
set.seed(123)  # Optional: for reproducibility
data$r <- ifelse(data$income < 150000, runif(nrow(data)), NA)

# Sort by the random number (NA values will go to the end by
default)
data <- data[order(data$r), ]

# Keep the first 100 observations
data_subset <- head(data, 100)

#eq(4.1a);
reg1 <- lm(income ~ educ, data=data)
summary(reg1)
#eq(4.1b);
reg2 <- lm(income ~ educ + afqt, data=data)
summary(reg2)
#eq(4.1c);
reg3 <- lm(educ ~ afqt, data=data)
summary(reg3)
data$ehat1<-predict(reg3)
#eq(4.1d);
data$adjeduc<- data$educ- data$ehat1
#eq(4.1e);
reg4 <- lm(educ ~ adjeduc, data=data)
summary(reg4)
#for standard deviations in Figure 4.3;
sd(data$educ)
sd(data$adjeduc)
```

Section 4.3

```
#For Table 4.2 & Fig. 4.4;
eval = read.csv("instructor-eval-ch4.csv", header = TRUE)
```

```
#eq (4.5)
reg1 <- lm(e ~ cs, data=eval)
summary(reg1)
#eq (4.6)
reg2 <- lm(e ~ cs + p2 + p3 + p4, data=eval)
summary(reg2)
#eq (4.7a)
reg4a <- lm(e ~ cs, data=subset(eval, p1==1))
summary(reg4a)
#eq (4.7b)
reg4b <- lm(e ~ cs, data=subset(eval, p2==1))
summary(reg4b)
#eq (4.7c)
reg4c <- lm(e ~ cs, data=subset(eval, p3==1))
summary(reg4c)
#eq (4.7d)
reg4d <- lm(e ~ cs, data=subset(eval, p4==1))
summary(reg4d)

#get variance of cs within each professor quartile for Table
4.2;
var(eval$cs[eval$p1==1])
var(eval$cs[eval$p2==1])
var(eval$cs[eval$p3==1])
var(eval$cs[eval$p4==1])
```

Section 4.5

```
#Figure 4.9;
#eq(4.15)
dominos = read.csv("dominos.csv", header = TRUE)
reg1 <- lm(d5 ~ d1, data=dominos)
summary(reg1)
#eq(4.10 & 4.11)
reg2 <- lm(d5 ~ d1 + d3, data=dominos)
summary(reg2)
```

CHAPTER 5

Section 5.1.1

```
#for Table 5.1;

# Load required package
library(writexl)

# Define a function to replicate the Stata loop for a given
sample size
run_simulation <- function(n, filename) {
  set.seed(123)  # Optional: makes results reproducible

# Create the initial dataset
  data <- data.frame(
    y1 = runif(n),
    y2 = runif(n, 0, 3),
    x  = runif(n)
  )

# Prepare list to store regression outputs
  results_list <- vector("list", 50)

# Repeat 50 regressions as in Stata
  for (i in 1:50) {
    data$y1 <- runif(n)
    data$x  <- runif(n)

    model <- lm(y1 ~ x, data = data)

# Store regression coefficients and standard errors
    results_list[[i]] <-
as.data.frame(summary(model)$coefficients)
    results_list[[i]]$iteration <- i
  }

# Combine results into one data frame
  all_results <- do.call(rbind, results_list)

# Write to Excel file (like outreg2)
  write_xlsx(all_results, filename)
}

# Run for three sample sizes
run_simulation(100,   "imprecision100.xlsx")
run_simulation(1000,  "imprecision1000.xlsx")
```

```
run_simulation(10000, "imprecision10000.xlsx")
```

Section 5.1.2

```
#for Table 5.3;
income = read.csv("income.csv", header = TRUE)
reg1 <- lm(income ~ educ + afqt + ageyears + black + hisp +
mom_hs + mom_coll + dad_hs + dad_coll, data=income)
summary(reg1)
```

Section 5.1.3

```
#for Figure 5.1;
tv1 = read.csv("tv-bmi-Fig5.1-top.csv", header = TRUE)
reg1 <- lm(bmi ~ tv, data=tv1)
summary(reg1)
tv2 = read.csv("tv-bmi-Fig5.1-middle.csv", header = TRUE)
reg2 <- lm(bmi ~ tv, data=tv2)
summary(reg2)
tv3 = read.csv("tv-bmi-Fig5.1-bottom.csv", header = TRUE)
reg3 <- lm(bmi ~ tv, data=tv3)
summary(reg3)
```

Section 5.3.6.1

```
#for equation (5.9) testing for joint significance of 4 parents'
education variables;
install.packages("car")
library(car)
income = read.csv("income.csv", header = TRUE)
reg1 <- lm(income ~ educ + afqt + ageyears + black + hisp +
mom_hs + mom_coll + dad_hs + dad_coll, data=income)
#testing for joint significance of 4 parents' education
variables;
linearHypothesis(reg1, c("mom_hs=0", "mom_coll=0", "dad_hs=0",
"dad_coll=0"))
#testing for joint significance of 2 mothers' education
variables;
linearHypothesis(reg1, c("mom_hs=0", "mom_coll=0"))
```

Section 5.4.2 (heteroskedasticity)

```
install.packages("car")
library(car)
income = read.csv("c:/data/book/text_data/income.csv", header =
TRUE)
reg12 <- lm(income ~ educ + afqt, data=income)
#test for heteroskedasticity with respect to the fitted value
(the Breusch-Pagan test);
ncvTest(reg12)
#test for heteroskedasticity with respect to educ
ncvTest(reg12, ~ educ)
#test for heteroskedasticity with respect to afqt
ncvTest(reg12, ~ afqt)
```

Section 5.4.3 (correlated observations)

```
#Code for correction for clustered standard errors:
#mydata = dataset
#class = variable identifying which observations might be
correlated with each other
#reg1 = the original uncorrected regression

reg1$clse <-cluster.vcov(reg1, mydata$class)
coeftest(reg1, reg1$clse)
```

Section 5.8

```
#FOR TABLE 5.8 - MOTIVATION-INCOME EXAMPLE;

# Load required package for Excel export
library(writexl)

set.seed(9254)

# Create initial dataset with 1000 observations
n <- 1000
data <- data.frame(
  id = 1:n,
  type = ifelse(1:n > 250, 2, 1),
  income = runif(n, 30000, 100000),
  motivation = rnorm(n)
)
```

```
# Adjust income for type == 1
data$income[data$type == 1] <- data$income[data$type == 1] +
                               5000 * data$motivation[data$type
== 1]

# Prepare a list to store regression coefficients
results_list <- vector("list", 100)

# Repeat 100 regressions, regenerating random variables each
time
for (i in 1:100) {
  data$income <- runif(n, 30000, 100000)
  data$motivation <- rnorm(n)
  data$income[data$type == 1] <- data$income[data$type == 1] +
                                 5000 *
data$motivation[data$type == 1]

  # Run the regression (equivalent to Stata's "reg income
motivation")
  model <- lm(income ~ motivation, data = data)

  # Store coefficients, standard errors, t-values, and p-values
  coef_table <- as.data.frame(summary(model)$coefficients)
  coef_table$iteration <- i
  results_list[[i]] <- coef_table
}

# Combine all results into a single data frame
all_results <- do.call(rbind, results_list)

# Export results to Excel (equivalent to "outreg2 using
motivation-income.xls, noast")
write_xlsx(all_results, "motivation-income.xlsx")
```

CHAPTER 6

Section 6.4.1

```
#Table 6.2;

clear
set obs 1000
g type="A"
replace type="B" if _n>500

g gpa = rnormal()
g gpa_f1 = rnormal()
g gpa_f2 = gpa_f1

replace gpa=gpa+0.5*gpa_f1 if type=="A"
replace gpa_f2=gpa*.8 if type=="B"

*RANDOM WORLD--gpa is not subject to reverse causality;
reg gpa gpa_f1, rob

*NON-RANDOM WORLD--gpa is subject to reverse causality;
reg gpa gpa_f2, rob

*repeated regressions to obtain average tendency;
forvalues i = 1/50 {
     replace gpa = rnormal()
     replace gpa_f1 = rnormal()
     replace gpa_f2 = gpa_f1
     replace gpa=gpa+0.5*gpa_f1 if type=="A"
     replace gpa_f2=gpa*.8 if type=="B"
     reg gpa gpa_f1, rob
       outreg2 using peer1.xls, noast
     reg gpa gpa_f2, rob
       outreg2 using peer2.xls, noast
  }
```

#Table 6.3 (omega3-depression example);

```
clear
set ob 1000
g type="A"
replace type="B" if _n>300

g dep = rnormal()
g omega1 = rnormal()
```

```
g omega2 = omega1
replace dep=dep-0.2*omega1 if type=="A"
replace omega2=dep*.3 if type=="B"

*RANDOM WORLD--gpa is not subject to reverse causality;
reg dep omega1, rob

*NON-RANDOM WORLD--gpa is subject to reverse causality;
reg dep omega2, rob

*repeated regressions to obtain average tendency;
forvalues i = 1/50 {
     replace dep = rnormal()
     replace omega1 = rnormal()
     replace omega2 = omega1
     replace dep=dep-0.2*omega1 if type=="A"
     replace omega2=dep*.3 if type=="B"
     *RANDOM WORLD--depression is not subject to reverse
causality;
     reg dep omega1, rob
     outreg2 using omega-dep1.xls, noast
     *NON-RANDOM WORLD--depression is subject to reverse
causality;
     reg dep omega2, rob
     outreg2 using omega-dep2.xls, noast
  }
```

Section 6.5.3

```
#demonstrating omitted-variables bias;
inc = read.csv("income.csv", header = TRUE)
#eq(2.4b);
reg5 <- lm(income ~ educ, data=inc)
summary(reg5)
#eq(2.21);
reg6 <- lm(income ~ educ + afqt, data=inc)
summary(reg6)
*eq(6.9);
reg7 <- lm(afqt ~ educ, data=inc)
summary(reg7)
```

Section 6.5.8

```
#Table 6.5: Omit-Factors-Bias from common factors;
clear
```

```
set obs 1000
g x2 = rnormal()
g x1 = rnormal()+.5*x2
g y = rnormal()+x1+2*x2

forvalues i = 1/50 {
     replace x2 = rnormal()
     replace x1 = rnormal()+.5*x2
     replace y = rnormal()+x1+2*x2
     reg x2 x1, rob
     outreg2 using commfactors-x2x1-1000.xls, noast dec(3)
     reg y x1 x2, rob
     outreg2 using commfactors-no-OFB-1000.xls, noast dec(3)
     reg y x1, rob
     outreg2 using commfactors-OFB-1000.xls, noast dec(3)
  }
```

Section 6.5.8

```
#THIS USES TIME-SERIES, WHICH WE ARE BORROWING FROM CHAPTER 10;
#Table 6.6;
install.packages("dynlm")
library(dynlm)
taxdata = read.csv("taxdata.csv", header = TRUE)
tax <- ts(taxdata$taxrate)
ur <- ts(taxdata$urate)
rgdp <- ts(taxdata$rgdp)

reg3 <- dynlm(ur ~ L(ur,1) + tax)
reg4 <- dynlm(rgdp ~ L(rgdp,1) + tax)
summary(reg3)
summary(reg4)
```

```
#Tables 6.7 & 6.8: Omit-Factors-Bias from INCIDENTAL
CORRELATION;
# Load necessary libraries
library(writexl)   # For exporting results
library(sandwich)  # For robust covariance estimation
library(lmtest)    # For coeftest()

set.seed(9254)
```

```
# --- Define simulation function ---
run_simulation <- function(n, prefix) {
  # Create initial dataset
  data <- data.frame(
    x1 = rnorm(n),
    x2 = rnorm(n)
  )
  data$y <- rnorm(n) + data$x1 + 2 * data$x2

  # Prepare lists to collect results from each regression type
  results_x2x1 <- vector("list", 50)
  results_y_x1x2 <- vector("list", 50)
  results_y_x1 <- vector("list", 50)

  # Repeat regressions 50 times
  for (i in 1:50) {
    data$x1 <- rnorm(n)
    data$x2 <- rnorm(n)
    data$y  <- rnorm(n) + data$x1 + 2 * data$x2

    # 1. reg x2 x1, rob
    model1 <- lm(x2 ~ x1, data = data)
    rob1 <- coeftest(model1, vcov = vcovHC(model1, type =
"HC1"))
    df1 <- as.data.frame(rob1)
    df1$iteration <- i
    results_x2x1[[i]] <- df1

    # 2. reg y x1 x2, rob
    model2 <- lm(y ~ x1 + x2, data = data)
    rob2 <- coeftest(model2, vcov = vcovHC(model2, type =
"HC1"))
    df2 <- as.data.frame(rob2)
    df2$iteration <- i
    results_y_x1x2[[i]] <- df2

    # 3. reg y x1, rob
    model3 <- lm(y ~ x1, data = data)
    rob3 <- coeftest(model3, vcov = vcovHC(model3, type =
"HC1"))
    df3 <- as.data.frame(rob3)
    df3$iteration <- i
    results_y_x1[[i]] <- df3
  }

  # Combine results across iterations
  combined_x2x1  <- do.call(rbind, results_x2x1)
```

```
  combined_yx1x2 <- do.call(rbind, results_y_x1x2)
  combined_yx1   <- do.call(rbind, results_y_x1)

  # Write each to Excel file (like outreg2)
  write_xlsx(combined_x2x1,  paste0(prefix, "-x1-x2.xlsx"))
  write_xlsx(combined_yx1x2, paste0(prefix, "-y-x1-x2.xlsx"))
  write_xlsx(combined_yx1,   paste0(prefix, "-y-x1.xlsx"))
}

# --- Run simulations for the three sample sizes ---
run_simulation(100,   "incidental-100")
run_simulation(1000,  "incidental-1000")
run_simulation(10000, "incidental-10000")
```

Section 6.6.6

```
#TABLE 6.10 - DEMONSTRATING SELF-SELECTION BIAS;
# Load required packages
library(writexl)   # for writing to Excel
library(sandwich)  # for robust standard errors
library(lmtest)    # for coeftest()

set.seed(9254)

# --- Create base dataset (1000 observations) ---
n <- 1000
data <- data.frame(
  id = 1:n,
  type = cut(1:n,
             breaks = c(0, 250, 500, 750, 1000),
             labels = c(1, 2, 3, 4))
)

data$type <- as.numeric(as.character(data$type))

# Generate variables
data$r <- runif(n)
data$x1 <- data$r * data$type
data$y <- rnorm(n)

# Group-specific effects on y
data$y[data$type == 2] <- data$y[data$type == 2] + 0.2 *
data$x1[data$type == 2]
data$y[data$type == 3] <- data$y[data$type == 3] + 0.4 *
data$x1[data$type == 3]
```

```
data$y[data$type == 4] <- data$y[data$type == 4] + 0.6 *
data$x1[data$type == 4]

# Prepare lists for results
results_no_type <- vector("list", 50)
results_with_type <- vector("list", 50)

# --- Loop for repeated regressions ---
for (i in 1:50) {
  # Regenerate random variables
  data$r <- runif(n)
  data$x1 <- data$r * data$type
  data$y <- rnorm(n)

  data$y[data$type == 2] <- data$y[data$type == 2] + 0.2 *
data$x1[data$type == 2]
  data$y[data$type == 3] <- data$y[data$type == 3] + 0.4 *
data$x1[data$type == 3]
  data$y[data$type == 4] <- data$y[data$type == 4] + 0.6 *
data$x1[data$type == 4]

  # --- Regression 1: y ~ x1 (no type variable)
  model1 <- lm(y ~ x1, data = data)
  rob1 <- coeftest(model1, vcov = vcovHC(model1, type = "HC1"))
  df1 <- as.data.frame(rob1)
  df1$iteration <- i
  results_no_type[[i]] <- df1

  # --- Regression 2: y ~ x1 + type (as factor)
  model2 <- lm(y ~ x1 + factor(type), data = data)
  rob2 <- coeftest(model2, vcov = vcovHC(model2, type = "HC1"))
  df2 <- as.data.frame(rob2)
  df2$iteration <- i
  results_with_type[[i]] <- df2
}

# --- Combine results across all iterations ---
SSB_no_type_results <- do.call(rbind, results_no_type)
SSB_with_type_results <- do.call(rbind, results_with_type)

# --- Export to Excel (like outreg2) ---
write_xlsx(SSB_no_type_results, "SSB-no-type.xlsx")
write_xlsx(SSB_with_type_results, "SSB-with-type.xlsx")
```

Section 6.7.2

```
#for Table 6.11, demonstrating the bias from measurement error;
inc = read.csv("income.csv", header = TRUE)

inc$collx=income$coll
inc$random=runif(2772,0,1)
inc$collx[inc$random<.1] <- 1-inc$coll

reg8 <- lm(income ~ coll + afqt + ageyears + black + hisp +
mom_hs + mom_coll + dad_hs + dad_coll, data=inc)
summary(reg8)
reg9 <- lm(income ~ collx + afqt + ageyears + black + hisp +
mom_hs + mom_coll + dad_hs + dad_coll, data=inc)
summary(reg9)
#[you will get different results for the 2nd model due to the
randomization];
```

Section 6.7.3

```
** FOR TABLE 6.12 - DEMONSTRATING THE EXTENT OF MEAS ERROR BIAS;

# Load packages
library(writexl)
library(sandwich)
library(lmtest)

set.seed(9254)

# --- Step 1: Base data generation ---
n <- 1000
data <- data.frame(
  x1 = rnorm(n),
  y = rnorm(n),
  r = runif(n)
)
data$y <- data$y + data$x1

# Create 9 measured versions of x1
for (j in 1:9) {
  data[[paste0("x1me", j)]] <- data$x1
}

# --- Helper function to apply measurement error ---
```

```
apply_me_error <- function(data) {
  r <- data$r
  x1 <- data$x1

  # Magnitude (.5, 1, 2) × probability (.1, .2, .5)
  # Group 1-3: 10% tail changes
  data$x1me1 <- ifelse(r < 0.1, x1 + 0.5,
                       ifelse(r > 0.9, x1 - 0.5, x1))
  data$x1me2 <- ifelse(r < 0.1, x1 + 1,
                       ifelse(r > 0.9, x1 - 1, x1))
  data$x1me3 <- ifelse(r < 0.1, x1 + 2,
                       ifelse(r > 0.9, x1 - 2, x1))

  # Group 4-6: 20% tail changes
  data$x1me4 <- ifelse(r < 0.2, x1 + 0.5,
                       ifelse(r > 0.8, x1 - 0.5, x1))
  data$x1me5 <- ifelse(r < 0.2, x1 + 1,
                       ifelse(r > 0.8, x1 - 1, x1))
  data$x1me6 <- ifelse(r < 0.2, x1 + 2,
                       ifelse(r > 0.8, x1 - 2, x1))

  # Group 7-9: 50% split changes
  data$x1me7 <- ifelse(r < 0.5, x1 + 0.5, x1 - 0.5)
  data$x1me8 <- ifelse(r < 0.5, x1 + 1, x1 - 1)
  data$x1me9 <- ifelse(r < 0.5, x1 + 2, x1 - 2)

  return(data)
}

# Apply the initial measurement error setup
data <- apply_me_error(data)

# --- Step 2: Loop through 100 simulations ---
# Initialize lists for results from 10 models
results <- vector("list", 10)
for (k in 1:10) results[[k]] <- list()

for (i in 1:100) {
  # Reapply measurement error each iteration
  data <- apply_me_error(data)

  # --- Run regressions ---
  # Model 1: y ~ x1 (true variable)
  model0 <- lm(y ~ x1, data = data)
  rob0 <- coeftest(model0, vcov = vcovHC(model0, type = "HC1"))
  df0 <- as.data.frame(rob0)
  df0$iteration <- i
```

```
  results[[1]][[i]] <- df0

  # Models 2-10: y ~ x1me1 ... x1me9
  for (m in 1:9) {
    model_m <- lm(reformulate(paste0("x1me", m), "y"), data =
data)
    rob_m <- coeftest(model_m, vcov = vcovHC(model_m, type =
"HC1"))
    df_m <- as.data.frame(rob_m)
    df_m$iteration <- i
    results[[m + 1]][[i]] <- df_m
  }
}

# --- Step 3: Combine and export results ---
for (m in 1:10) {
  combined <- do.call(rbind, results[[m]])
  if (m == 1) {
    file_name <- "meas-error-0.xlsx"
  } else {
    file_name <- paste0("meas-error-", m - 1, ".xlsx")
  }
  write_xlsx(combined, file_name)
}
```

Section 6.7.5

```
#Table 6.13;
eval = read.csv("instructor-eval-ch4.csv", header = TRUE)
eval$r<-runif(100,0,1)
eval$csx<- eval$cs+(eval$r<.1)*(-15)+(eval$r>.9)*(15)


reg1 <- lm(e ~ cs, data=eval)
summary(reg1)
reg2 <- lm(e ~ csx, data=eval)
summary(reg2)
reg3 <- lm(e ~ cs + p2 + p3 + p4, data=eval)
summary(reg3)
reg4 <- lm(e ~ csx + p2 + p3 + p4, data=eval)
summary(reg4)
#[you will get different results for the 2nd and 4th models due
to the randomization]
```

Section 6.8.4

```
#Table 6.14;
tv = read.csv("tv-bmi-med-factors.csv", header = TRUE)
reg1 <- lm(bmi ~ tv + junkfood, data=tv)
summary(reg1)
reg2 <- lm(bmi ~ tv, data=tv)
summary(reg2)
```

Section 6.10.1

```
#Fig 6.16 & Table 6.17;

eval = read.csv("instructor-eval-ch6.csv", header = TRUE)

reg1 <- lm(e ~ cs, data=eval)
summary(reg1)
reg2 <- lm(e ~ cs + p2 + p3 + p4, data=eval)
summary(reg2)
reg3a <- lm(e ~ cs, data=subset(eval, profnum==1)
summary(reg3a)
reg3b <- lm(e ~ cs, data=subset(eval, profnum==2))
summary(reg3b)
reg3c <- lm(e ~ cs, data=subset(eval, profnum==3))
summary(reg3c)
reg3d <- lm(e ~ cs, data=subset(eval, profnum==4))
summary(reg3d)

#get variance of cs within each professor for Tables 6.18 &
6.19;
var(eval$cs[eval$profnum==1])
var(eval$cs[eval$profnum==2])
var(eval$cs[eval$profnum==3])
var(eval$cs[eval$profnum==4])
```

Section 6.10.3

```
*Table 6.19;

# Load required libraries
library(dplyr)      # for grouped calculations
library(sandwich)   # for robust variance-covariance estimators
library(lmtest)     # for coeftest()
```

```
# --- 1. Read dataset ---
data = read.csv("instructor-eval-ch6.csv", header = TRUE)

# --- 2. Calculate std. deviation of cs for each professor ---
data <- data %>%
  group_by(profnum) %>%
  mutate(
    sd = sd(cs, na.rm = TRUE),
    var = sd^2,
    wt = 1 / var
  ) %>%
  ungroup()

# --- 3. Regression 1: unweighted with professor fixed effects -
--
model1 <- lm(e ~ cs + factor(profnum), data = data)
robust1 <- coeftest(model1, vcov = vcovHC(model1, type = "HC1"))

# --- 4. Regression 2: weighted by 1/var (analytic weights) ---
model2 <- lm(e ~ cs + factor(profnum), data = data, weights =
wt)
robust2 <- coeftest(model2, vcov = vcovHC(model2, type = "HC1"))

# --- 5. Display results ---
cat("\nUnweighted Regression (robust SEs):\n")
print(robust1)

cat("\nWeighted Regression (robust SEs):\n")
print(robust2)
```

Section 6.10.4

```
#Table 6.20;

# Load required packages
library(dplyr)      # for data manipulation
library(lmtest)     # for coeftest()
library(sandwich)   # for robust SEs
library(writexl)    # for exporting to Excel

# --- 1. Load the dataset ---
data = read.csv("bmi-tv-v3.csv", header = TRUE)

# --- 2. Run weighted regressions by race group (robust SEs) ---
race_levels <- sort(unique(data$race2))
```

```
results_by_race <- list()

for (r in race_levels) {
  sub <- subset(data, race2 == r)
  model <- lm(zbmi4 ~ tv4 + age4 + female + inc4_lt30 + inc4_gt70,
              data = sub, weights = wt4)
  rob <- coeftest(model, vcov = vcovHC(model, type = "HC1"))
  results_by_race[[as.character(r)]] <- rob
}

# --- 3. Pooled model with race indicators ---
model_full <- lm(zbmi4 ~ tv4 + age4 + female + inc4_lt30 + inc4_gt70 +
factor(race2),
                 data = data, weights = wt4)
rob_full <- coeftest(model_full, vcov = vcovHC(model_full, type =
"HC1"))

# --- 4. Compute residual-based correction weights ---

# Step 1: Predict tv4 controlling for covariates and race
model_tv <- lm(tv4 ~ age4 + female + inc4_lt30 + inc4_gt70 +
factor(race2),
               data = data)
data$tv4hat <- predict(model_tv)
data$tv4res <- data$tv4 - data$tv4hat

# Step 2: Compute sd, var, and correction weights within race
data <- data %>%
  group_by(race2) %>%
  mutate(sd = sd(tv4res, na.rm = TRUE),
         var = sd^2,
         rw = wt4 / var) %>%
  ungroup()

# --- 5. Weighted regressions (two versions) ---
# (a) Using original weights
model_wt <- lm(zbmi4 ~ tv4 + age4 + female + inc4_lt30 + inc4_gt70 +
factor(race2),
               data = data, weights = wt4)
rob_wt <- coeftest(model_wt, vcov = vcovHC(model_wt, type = "HC1"))

# (b) Using regression-corrected weights
model_rw <- lm(zbmi4 ~ tv4 + age4 + female + inc4_lt30 + inc4_gt70 +
factor(race2),
               data = data, weights = rw)
rob_rw <- coeftest(model_rw, vcov = vcovHC(model_rw, type = "HC1"))

# --- 6. Export both regression results to Excel (like outreg2) ---
results_list <- list(
  by_race = lapply(results_by_race, as.data.frame),
  full_weighted = as.data.frame(rob_wt),
```

```
  corrected_weighted = as.data.frame(rob_rw)
)

# Flatten and export
write_xlsx(
  list(
    "Race-specific models" = do.call(rbind, results_list$by_race),
    "Weighted (wt4)" = results_list$full_weighted,
    "Weighted (rw)" = results_list$corrected_weighted
  ),
  "tv-bmi-correction.xlsx"
)

# --- 7. Summary stats for Table 6.21 ---
summary_stats <- data %>%
  group_by(race2) %>%
  summarise(
    mean_tv4res = weighted.mean(tv4res, wt4, na.rm = TRUE),
    mean_wt4 = weighted.mean(wt4, wt4, na.rm = TRUE)
  )

print(summary_stats)
```

Section 6.13

```
*Table 6.23 - outliers;

# Load required libraries
library(dplyr)       # for data manipulation
library(sandwich)    # for robust covariance matrices
library(lmtest)      # for coeftest()
library(writexl)     # for Excel export

# --- 1. Load dataset ---
data = read.csv("home-prices.csv", header = TRUE)

# --- 2. First regression (robust SEs) ---
model_full <- lm(price ~ bedrooms + bathrooms + sqft + sqft_lot
+ view + waterfront,
                 data = data)
rob_full <- coeftest(model_full, vcov = vcovHC(model_full, type
= "HC1"))

# --- 3. Compute fitted values and standardized residuals ---
data$yhat <- predict(model_full)
data$res <- data$price - data$yhat
sd_res <- sd(data$res, na.rm = TRUE)
```

```
data$stdres <- data$res / sd_res

# --- 4. Compute percentiles (Stata's xtile equivalent) ---
data <- data %>%
  mutate(
    price_pct = ntile(price, 100),
    stdres_pct = ntile(stdres, 100)
  )

# --- 5. Store results for export ---
results <- list()
results[["All observations"]] <- as.data.frame(rob_full)

# --- 6. Exclude top price percentile (price_pct != 100) ---
model_price <- lm(price ~ bedrooms + bathrooms + sqft + sqft_lot
+ view + waterfront,
                  data = filter(data, price_pct != 100))
rob_price <- coeftest(model_price, vcov = vcovHC(model_price,
type = "HC1"))
results[["Exclude top price percentile"]] <-
as.data.frame(rob_price)

# --- 7. Exclude top stdres percentile (stdres_pct != 100) ---
model_resid <- lm(price ~ bedrooms + bathrooms + sqft + sqft_lot
+ view + waterfront,
                  data = filter(data, stdres_pct != 100))
rob_resid <- coeftest(model_resid, vcov = vcovHC(model_resid,
type = "HC1"))
results[["Exclude top stdres percentile"]] <-
as.data.frame(rob_resid)

# --- 8. Exclude both (price_pct != 100 & stdres_pct != 100) ---
model_both <- lm(price ~ bedrooms + bathrooms + sqft + sqft_lot
+ view + waterfront,
                 data = filter(data, price_pct != 100 &
stdres_pct != 100))
rob_both <- coeftest(model_both, vcov = vcovHC(model_both, type
= "HC1"))
results[["Exclude both"]] <- as.data.frame(rob_both)

# --- 9. Export results (like outreg2) ---
write_xlsx(results, "home-prices-outliers.xlsx")

# --- 10. Optional: tabulate price percentile counts (like tab
price_pct) ---
table(data$price_pct)
```

CHAPTER 7

Section 7.1.1 (for Table 7.2) – this is a simulation, for which your results will be different from those in the book … or you can use the data set, "insurance-prediction" to obtain the results in the book;

```
# Load required packages
library(dplyr)
library(lmtest)
library(sandwich)
library(writexl)

set.seed(123)

# --- 1. Create base dataset with 10,000 observations ---
n <- 10000
data <- data.frame(id = 1:n)

# --- 2. Generate x1 (accidents over 10 years) ---
data$x1 <- 0
data$x1[data$id > 9000] <- 1
data$x1[data$id > 9400] <- 2
# (Optional third category, same as Stata commented line)
# data$x1[data$id > 9800] <- 3

# Dummy indicators
data$x1_1 <- as.numeric(data$x1 == 1)
data$x1_2 <- as.numeric(data$x1 == 2)

# --- 3. Generate other variables ---
data$x2 <- as.numeric(runif(n) > 0.5)             # College
degree dummy
data$x3 <- runif(n, 20, 75)                       # Age
data$x4 <- runif(n, 40, 100) + 50 * data$x2       # Income
correlated with education

# --- 4. Construct latent index and outcome ---
data$yindex <- runif(n, 0, 0.2) +
               0.08 * data$x1_1 +
               0.12 * data$x1_2 -
               0.10 * data$x2 -
               0.002 * data$x3 -
               0.002 * data$x4

# Decile classification (like Stata's xtile)
```

```
data <- data %>% mutate(d = ntile(yindex, 10))

# Binary outcome: top decile indicator
data$y <- as.numeric(data$d == 10)

# --- 5. Rename variables for clarity ---
data <- data %>%
  rename(
    accidents10yrs = x1,
    colldeg = x2,
    age = x3,
    income = x4
  )

# --- 6. Run regressions (robust SEs) ---
# Model 1: without income
model1 <- lm(y ~ accidents10yrs + colldeg + age, data = data)
rob1 <- coeftest(model1, vcov = vcovHC(model1, type = "HC1"))

# Model 2: with income
model2 <- lm(y ~ accidents10yrs + colldeg + age + income, data =
data)
rob2 <- coeftest(model2, vcov = vcovHC(model2, type = "HC1"))

# --- 7. Export to Excel (similar to outreg2) ---
results <- list(
  "Model 1 (no income)" = as.data.frame(rob1),
  "Model 2 (with income)" = as.data.frame(rob2)
)

write_xlsx(results, "insurance-prediction.xlsx")
```

Section 7.2.2

```
#Table 7.3;
health50 = read.csv("health50_females.csv", header = TRUE)
reg1 <- lm(H50 ~ H40 + black + afqt, data=health50)
reg2 <- lm(H50 ~ H40, data=health50)
reg3 <- lm(H50 ~ black, data=health50)
reg4 <- lm(H50 ~ afqt, data=health50)

summary(reg1)
summary(reg2)
summary(reg3)
summary(reg4)
```

CHAPTER 8

Section 8.1

General commands to estimate fixed effects

Let's use the following set-up:

- *y* = dependent variable
- *x1* & *x2* are explanatory variables
- *group* = fixed-effects indicator for, say, 4 different groups
- *g1*, *g2*, *g3*, and *g4* are indicators for being in the 4 fixed-effects groups. A given observation would have a value of one for just one of these variables and a value of zero for the other three variables.

There is one main method for estimating fixed effects. And, there are at least two alternative methods that would give the same results, provided that there is no correction to the standard errors for heteroskedasticity or clustering. If so, then the main method is the optimal method.

```
library(plm)
reg1 <- plm(y ~ x1 x2, data=mydata, index=c("group"),
model="within")
```

(This is similar to equation (8.3).)

If there were not a heteroskedasticity correction and there were no clustering of standard errors, then there are three other approaches (below) that would give similar results. The **xtreg** approach is optimal because it properly adjusts the standard errors for these situations. But, other situations cannot be properly applied with the **xtreg** command. For example, if different weights are needed for each observation within a fixed-effects group, **xtreg** will not allow it, and one of the three alternative approaches below would need to be used.

Fixed-effects Alternative Method 1

```
alt1 <-lm(y ~ x1 + x2 + g2 + g3 + g4, data=mydata)
```

(This is similar to equation (8.4), controlling for courses (**g2** – **g4**), with **g1** excluded.)

Fixed-effects Alternative Method 2

```
alt2 <-lm(y ~ x1 + x2 + factor(group) - 1, data= mydata)
```

(This is also similar to equation (8.4).)

Two-way fixed effects

If you need to do have two sets of fixed effects (two-way fixed effects)—say, **group** and **year**—there is an R function (felm) that does not appear to work anymore. The command would be:

```
library(lfe)
fe2way <- felm(y ~ x1 + x2 + G(group) + G(year), data=mydata)
```

(See https://www.rdocumentation.org/packages/lfe/versions/1.6-1163/topics/felm for details.)

But, when attempting to estimate this model, R now says that the "G( ) syntax is deprecated," meaning that it is going to be replaced. I do not have an answer, so the easy solution would be a normal fixed effects model, with an alternative model for one of the fixed-effects groups, as follows:

```
reg1 <- plm(y ~ x1 x2 factor(year) - 1, data=mydata,
index=c("group"), model="within")
```

For interacted fixed effects

The best approach is to create a new fixed-effect-group indicator variable, and set the groups based on that variable. If the group has values with less than 3 digits, then we can do the following:

```
mydata$yeargroup <- mydata$year*100 + mydata$group
reg1 <- plm(y ~ x1 x2, data=mydata, index=c("yeargroup")
```

Correction for bias from having fixed effects and a lagged-dependent variable

This is the Arrellano-Bond correction, discussed in Section 8.1.

```
#coeftest(fixed, vcovHC(fixed, method = "arellano")
```

Section 8.2.2

```
#Table 8.2;
MJ = read.csv("MJ-UR-new.csv", header = TRUE)
#cross-section for 2006-07 data;
reg8 <- lm(mj0607 ~ ur0607, data=MJ, weights=sqrtpop2010)
```

```
#cross-section for 2009-10 data;
reg9 <- lm(mj0910 ~ ur0910, data=MJ, weights=sqrtpop2010)
#first-difference model;
reg10 <- lm(mjx ~ urx, data=MJ, weights=sqrtpop2010)
#mjx and urx are the changes from the first (2006-07) to second
(2009-10) period)
summary(reg8)
summary(reg9)
summary(reg10)
#can't do heteroskedasticity correction with weighted panel
models
```

Section 8.3

```
#notional example of Figure 8.3;
ivmodel <- ivreg(y ~ x1 + x2 | x2 + z, data=mydata)
```

The idea is that the explanatory variables in the second stage are put to the left of the divider in the variables list ("|"), and the explanatory variables in the first stage are to the right of the divider. So, we have

- x1 (the endogenous variable) being on the left but not the right
- z (the instrumental variable) being on the right but not the left.

What this code does is that it uses the predicted value of **x1** (rather than actual **x1**) based on variable **z** and **x2**. It is important to use the **ivreg** function rather than do the steps manually because the **ivreg** function automatically adjusts the standard errors based on the presence of using a fitted value for **x1**.

Afterwards, you would use "summary," or "coeftest" if you wanted to correct for heteroskedasticity.

Note that the first stage would need to be modeled separately if you wanted to see those results:

```
stage1 <- lm(x1 ~ x2 z, data=mydata)
```

Section 8.4.2

```
*TABLE 8.5 - DEMONSTRATING HOW 2SLS MODEL CORRECTS FOR
MEASUREMENT ERROR;
```

```
# Load required packages
library(AER)        # for ivreg()
library(lmtest)     # for coeftest()
library(sandwich)   # for robust vcov
library(writexl)    # for Excel export
library(dplyr)

set.seed(123)

# --- 1. Initialize base dataset ---
n <- 1000
data <- data.frame(id = 1:n)

# --- 2. Create initial variables ---
data$z <- rnorm(n)
data$x1 <- rnorm(n) + 0.3 * data$z
data$x1e <- data$x1
data$x1e[data$id <= 500] <- data$x1[data$id <= 500] - 1
data$x1e[data$id > 500]  <- data$x1[data$id > 500]  + 1
data$y <- rnorm(n) + data$x1

# --- 3. Prepare results containers ---
ols_results <- list()
iv_results  <- list()

# --- 4. Repeat simulation 100 times ---
for (i in 1:100) {
  # regenerate data each iteration
  data$z  <- rnorm(n)
  data$x1 <- rnorm(n) + 0.3 * data$z
  data$x1e[data$id <= 500] <- data$x1[data$id <= 500] - 1
  data$x1e[data$id > 500]  <- data$x1[data$id > 500]  + 1
  data$y  <- rnorm(n) + data$x1

  # OLS regression (robust SEs)
  ols <- lm(y ~ x1e, data = data)
  ols_rob <- coeftest(ols, vcov = vcovHC(ols, type = "HC1"))

  # IV regression (robust SEs)
  iv <- ivreg(y ~ x1e | z, data = data)
  iv_rob <- coeftest(iv, vcov = vcovHC(iv, type = "HC1"))

  # Store results
  ols_results[[paste0("Iteration_", i)]] <-
as.data.frame(ols_rob)
```

```
  iv_results[[paste0("Iteration_", i)]]  <-
as.data.frame(iv_rob)
}

# --- 5. Export results to Excel (like outreg2) ---
write_xlsx(ols_results, "iv1.xlsx")
write_xlsx(iv_results, "iv2.xlsx")
```

Section 8.6.4

PLEASE NOTE THAT, FOR VARIOUS REASONS, I AM ONLY SHARING THE CODE AND DATA FOR THE U.S. ANALYSIS AND NOT FOR THE EUROPE ANALYSIS.

```
# Required packages
library(plm)         # for panel data models (FE, Arellano-Bond,
etc.)
library(lmtest)      # for coeftest()
library(sandwich)    # for robust vcov estimators
library(writexl)     # optional, for exporting results

# --- 1. Load data ---
data = read.csv("us-overdose-deaths.csv", header = TRUE)

# --- 2. Declare panel structure ---
# fips = panel (state), year = time
pdata <- pdata.frame(data, index = c("fips", "year"))

# ===========================
# TABLE 8.6
# ===========================

# --- (a) Fixed Effects with Year Dummies ---
# Equivalent to: xtreg rate ur y15-y22, fe rob
fe_model <- plm(rate ~ ur + y15 + y16 + y17 + y18 + y19 + y20 +
y21 + y22,
                data = pdata, model = "within")
rob_fe <- coeftest(fe_model, vcov = vcovHC(fe_model, type =
"HC1"))

# --- (b) Arellano-Bond (difference GMM) ---
# Equivalent to: xtabond rate ur L.rate y15-y22, rob twostep
ab_model <- pgmm(
  formula = rate ~ lag(rate, 1) + ur + y15 + y16 + y17 + y18 +
y19 + y20 + y21 + y22 |
```

```
    lag(rate, 2:99),
  data = pdata,
  effect = "individual",
  model = "twosteps",
  transformation = "d"
)
summary(ab_model, robust = TRUE)

# --- (c) Arellano-Bover / Blundell-Bond (System GMM) ---
# Equivalent to: xtabond2 rate L.rate ur y15-y22, gmm(L.rate,
lag(2 .)) iv(ur) twostep
#                and collapse version

# Using pgmm for "system" GMM with appropriate instruments
sysgmm_model <- pgmm(
  formula = rate ~ lag(rate, 1) + ur + y15 + y16 + y17 + y18 +
y19 + y20 + y21 + y22 |
    lag(rate, 2:99),
  data = pdata,
  effect = "individual",
  model = "twosteps",
  transformation = "ld"  # 'ld' = system GMM (levels +
differences)
)
summary(sysgmm_model, robust = TRUE)

# Reduced instrument count version (collapsed instruments are
not explicit in R's pgmm;
# you would use fewer lag terms to achieve similar reduction)
sysgmm_reduced <- pgmm(
  formula = rate ~ lag(rate, 1) + ur + y15 + y16 + y17 + y18 +
y19 + y20 + y21 + y22 |
    lag(rate, 2:4),
  data = pdata,
  effect = "individual",
  model = "twosteps",
  transformation = "ld"
)
summary(sysgmm_reduced, robust = TRUE)

# ==========================
# TABLE 8.7
# ==========================

# Restrict to year >= 2015
pdata_15 <- pdata[pdata$year >= 2015, ]
```

```
# (a) Fixed Effects from 2015+
fe_2015 <- plm(rate ~ ur + y15 + y16 + y17 + y18 + y19 + y20 +
y21 + y22,
               data = pdata_15, model = "within")
rob_fe_2015 <- coeftest(fe_2015, vcov = vcovHC(fe_2015, type =
"HC1"))

# (b) Fixed Effects with Lagged Dependent Variable
fe_ldep <- plm(rate ~ ur + lag(rate, 1) + y15 + y16 + y17 + y18
+ y19 + y20 + y21 + y22,
               data = pdata, model = "within")
rob_fe_ldep <- coeftest(fe_ldep, vcov = vcovHC(fe_ldep, type =
"HC1"))

# --- 8. Export all results (optional) ---
results <- list(
  "FE (all years)" = as.data.frame(rob_fe),
  "Arellano-Bond" = as.data.frame(summary(ab_model, robust =
TRUE)$coefficients),
  "System GMM (full)" = as.data.frame(summary(sysgmm_model,
robust = TRUE)$coefficients),
  "System GMM (reduced)" = as.data.frame(summary(sysgmm_reduced,
robust = TRUE)$coefficients),
  "FE (>=2015)" = as.data.frame(rob_fe_2015),
  "FE + lag(rate)" = as.data.frame(rob_fe_ldep)
)

write_xlsx(results, "us-overdose-deaths-results.xlsx")
```

CHAPTER 9

Section 9.2.4

```
#Table 9.1;
MJ2000 = read.csv("MJ2000.csv", header = TRUE)
install.packages("mfx")
library(mfx)
#OLS
reg1 <-
lm(mj30~male+age+black+hisp+other+one_bio_parent+one_bio_other_p
arent+othersit, data=MJ2000)
coeftest(reg1, vcovHC, type="HC0")

#probit
reg2 <-
glm(mj30~male+age+black+hisp+other+one_bio_parent+one_bio_other_
parent+othersit, family=binomial(link="probit"), data=MJ2000)
coeftest(reg2, vcovHC, type="HC0")
#marginal effects
probitmfx(reg2, data=MJ2000, atmean = TRUE, robust = TRUE)

#logit and marginal effects
reg3 <-
glm(mj30~male+age+black+hisp+other+one_bio_parent+one_bio_other_
parent+othersit, family=binomial(link="logit"), data=MJ2000)
coeftest(reg3, vcovHC, type="HC0")
logitmfx(reg3, data=MJ2000, atmean = TRUE, robust = TRUE)


#Table 9.2;
reg3 <-
glm(mj30~male+age+black+hisp+other+one_bio_parent+one_bio_other_
parent+othersit, family=binomial(link="logit"), data=MJ2000)
coeftest(reg3, vcovHC, type="HC0")
logitmfx(reg3, data=MJ2000, atmean = TRUE, robust = TRUE)
logit mj30 $xvars, rob or
#odds ratios
logitor(reg3, data=MJ2000, robust = TRUE)
```

Section 9.3

```
install.packages("oglmx")
library(oglmx)
h40 = read.csv("health40_ordered.csv", header = TRUE)
#delete observations with missing data;
```

```
na.omit(h40)
#Table 9.3 ;
reg4 <-oglmx(H40 ~ educ+afqt+black+hisp+male, data = h40,
link="probit", constantMEAN = FALSE, constantSD =
FALSE,delta=0,threshparam = NULL)
summary(reg4)

#Table 9.4 ;
margins.oglmx(reg4,ascontinuous = TRUE)
```

Section 9.6

```
#Table 9.6--for count models;

numbirths = read.csv("numbirths.csv", header = TRUE)

#negative binomial

install.packages("MASS")
install.packages("mfx")
library(MASS)
library(mfx)
reg5 <- glm.nb(formula = numkids84 ~ exp_educ + afqt + black +
hisp, data = numbirths, init.theta = 1.03, link = log)
coeftest(reg5, vcovHC, type="HC0")
#marginal effects
negbinmfx(reg5, data=numbirths, robust = TRUE)

#poisson model
install.packages("MASS")
library(MASS)
reg6 <- glm(numkids84 ~ exp_educ + afqt + black + hisp,
family="poisson", data=numbirths)
coeftest(reg6, vcovHC, type="HC0")
#marginal effects
poissonmfx(reg6, data=numbirths, robust = TRUE)

reg7 <- lm(numkids84 ~ exp_educ + afqt + black + hisp,
data=numbirths)
coeftest(reg7, vcovHC, type="HC0")
```

Section 9.8

```
** DEMONSTRTAING QUANTILE REGS ... TABLE 9.8;

# Load required libraries
library(quantreg)    # For quantile regression
library(lmtest)      # For robust SEs with coeftest()
library(sandwich)    # For robust vcov matrices
library(writexl)     # For Excel export

# --- 1. Load data ---
data = read.csv("income.csv", header = TRUE)

# --- 2. OLS regression (equivalent to reg) ---
ols_model <- lm(income ~ educ + afqt, data = data)
ols_rob <- coeftest(ols_model, vcov = vcovHC(ols_model, type =
"HC1"))

# --- 3. Quantile regressions (qreg) ---
q25_model <- rq(income ~ educ + afqt, data = data, tau = 0.25)
q50_model <- rq(income ~ educ + afqt, data = data, tau = 0.50)
q75_model <- rq(income ~ educ + afqt, data = data, tau = 0.75)

# --- 4. Summaries with standard errors ---
# For quantile regressions, use bootstrapped SEs (analogous to
Stata’s default)
q25_sum <- summary(q25_model, se = "boot", R = 500)
q50_sum <- summary(q50_mode
```

CHAPTER 10

Section 10.1

```
install.packages("dynlm")
library(dynlm)

#Figure 10.1;
gas = read.csv("gasprices_1994_2003.csv", header = TRUE)
gas$spring=as.numeric(gas$qtr==2)
gas$summer=as.numeric(gas$qtr==3)
gas$fall=as.numeric(gas$qtr==4)

price <- ts(gas$price, start=c(1), end=c(40))
lagprice <- ts(gas$lagprice, start=c(1), end=c(40))
spring <- ts(gas$spring, start=c(1), end=c(40))
summer <- ts(gas$summer, start=c(1), end=c(40))
fall <- ts(gas$fall, start=c(1), end=c(40))
trend <- ts(gas$time, start=c(1), end=c(40))


reg1 <- lm(price ~ trend)
summary(reg1)
yhat1 <- fitted(reg1)

reg2 <- lm(price ~ spring + summer + fall)
summary(reg2)
yhat2 <- fitted(reg2)

reg3 <- lm(price ~ lagprice)
summary(reg3)
yhat3 <- fitted(reg3)

reg3 <- lm(price ~ lagprice)

reg4 <- lm(price ~ trend  + spring + summer + fall + lagprice)
summary(reg4)
res <- resid(reg4)

comb <- cbind(yhat1, yhat2, yhat3, res)
```

Section 10.3

```
#Table 10.1;
```

```
install.packages("stats4")
library(stats4)
install.packages("car")
library(car)

j = read.csv("japan-n225-ur-wine.csv", header = TRUE)
ur <- ts(j$ur, start=c(11), end=c(73))
reg1 <- arima(ur, order = c(1,0,0))
reg2 <- arima(ur, order = c(2,0,0))
reg3 <- arima(ur, order = c(3,0,0))
reg4 <- arima(ur, order = c(4,0,0))

# The heteroskedasticity correction is not feasible with arima
function,
#  so the results will be a little different from the textbook.
#  Also, the sample isn't limited to start in period 5 in these
models.
summary(reg1)
summary(reg2)
summary(reg3)
summary(reg4)
BIC(reg1)
BIC(reg2)
BIC(reg3)
BIC(reg4)
AIC(reg1)
AIC(reg2)
AIC(reg3)
AIC(reg4)
#AICc is calculated by   AICc = AIC + (2k^2+2k)/(n-k-1)
#k = #parameters to be estimated



*Table 10.2;
library(lmtest)
spurs = read.csv("spurs_win_pct_for_R.csv", header = TRUE)
spurs.ts <- ts(spurs, start=1977)
winpct <- ts(spurs$winpct, start=1977)
change <- ts(spurs$change, start=1978)
reg1 <- dynlm(winpct ~L(winpct,1))
reg2 <- dynlm(change ~ L(change,1))
reg3 <- lm(change ~ lagchange, data=subset(spurs, spurs$year !=
1998))
coeftest(reg1, vcovHC, type="HC0")
coeftest(reg2, vcovHC, type="HC0")
coeftest(reg3, vcovHC, type="HC0")
```

Section 10.4

```
#Table 10.3;

#FOR TABLE 10.3;
install.packages("dynlm")
library(dynlm)
j = read.csv("japan-n225-ur-wine.csv", header = TRUE)
ur <- ts(j$ur, start=c(11), end=c(73))
n225 <- ts(j$n225, start=c(11), end=c(73))

reg11 <- dynlm(ur ~ L(n225,1))
reg12 <- dynlm(ur ~ L(n225,1) + L(n225,2))
reg13 <- dynlm(ur ~ L(n225,1) + L(n225,2) + L(n225,3))
reg14 <- dynlm(ur ~ L(n225,1) + L(n225,2) + L(n225,3) +
L(n225,4))
reg15 <- dynlm(ur ~ L(ur,1) + n225)

coeftest(reg11, vcovHC, type="HC0")
coeftest(reg12, vcovHC, type="HC0")
coeftest(reg13, vcovHC, type="HC0")
coeftest(reg14, vcovHC, type="HC0")
coeftest(reg15, vcovHC, type="HC0")

AIC(reg11)
AIC(reg12)
AIC(reg13)
AIC(reg14)

BIC(reg11)
BIC(reg12)
BIC(reg13)
BIC(reg14)


#Table 10.4 -- Granger Causality;
library(sandwich)
j = read.csv("japan-n225-ur-wine.csv", header = TRUE)
ur <- ts(j$ur, start=c(11), end=c(73))
n225 <- ts(j$n225, start=c(11), end=c(73))

reg16 <- dynlm(n225 ~ L(n225,1) + L(ur,1))
coeftest(reg16, vcovHC, type="HC0")
```

```
#The NeweyWest function gives the covariance matrix. Take the
square root
# of the diagonal elements of the covariance matrix to obtain
the standard errors.
NeweyWest(reg16)
```

Section 10.5

```
#Durbin-Watson test:
# After a time-series regression (regx):  dwtest(regx)

#Breusch-Godfrey test:
#After a time-series regression (regx): bgtest(regx)

*test after Model (1) from Table 10.4;
reg16 <- dynlm(n225 ~ L(n225,1) + L(ur,1))
#Durbin-Watson Test
dwtest(reg16)
bgtest(reg16)
bgtest(reg16, order=2)
```

Section 10.6

```
install.packages("tseries")
library(tseries)

#Table 10.5 (on Dickey-Fuller tests);
#From output, Type 2 with 0 lags gives result in book;

sp500a = read.csv("sp500_oil_1990_1994.csv", header = TRUE)
sp500 <- ts(sp500a$sp500)
adf.test(sp500a$sp500)
oil <- ts(sp500a$oil)
adf.test(oil)

sp500b = read.csv("sp500_oil_2010_2014.csv", header = TRUE)
sp500 <- ts(sp500b$sp500)
adf.test(sp500)
oil <- ts(sp500b$oil)
adf.test(oil)
```

```
#Table 10.6;
sp500a = read.csv("sp500_oil_1990_1994.csv", header = TRUE)
oil90 <- ts(sp500a$oil)
sp500x90 <- ts(sp500a$sp500)
reg18 <- dynlm(sp500x90 ~ L(sp500x90,1) + oil90)
reg19 <- dynlm(diff(sp500x90, lag=1) ~ diff(oil90, lag=1))
coeftest(reg18, vcovHC, type="HC0")
coeftest(reg19, vcovHC, type="HC0")

sp500b = read.csv("sp500_oil_2010_2014.csv", header = TRUE)
oil00 <- ts(sp500b$oil)
sp500x00 <- ts(sp500b$sp500)
reg20 <- dynlm(sp500x00 ~ L(sp500x00,1) + oil00)
reg21 <- dynlm(diff(sp500x00, lag=1) ~ diff(oil00, lag=1))
coeftest(reg20, vcovHC, type="HC0")
coeftest(reg21, vcovHC, type="HC0")
```

Section 10.7

```
#Table 10.7;
install.packages("aTSA")
library(aTSA)
install.packages("vars")
library(vars)

j = read.csv("japan-n225-ur-wine.csv", header = TRUE)
ur <- ts(j$ur, start=c(11), end=c(73))
n225 <- ts(j$n225, start=c(11), end=c(73))

#preliminary nonstationarity tests;
adf.test(j$ur)
adf.test(j$n225)

#VAR MODEL
install.packages("vars")
library(vars)
j2<-subset(j,year>=1960)
x = cbind(j2$ur, j2$n225)
fitvar1 <- VAR(x,p=1, type="const")
summary(fitvar1)
#note that (type="both") in the VAR function would include both
a constant and a trend;
```

Section 10.8

```
#Prais-Winsten transformation
#install.packages("prais")
#library(prais)
#prais.winsten(formula, data, iter = 50, rho = 0, tol = 1e-08)
#from https://cran.r-project.org/web/packages/prais/prais.pdf

#Table 10.8;
sp500b = read.csv("sp500_oil_2010_2014.csv", header = TRUE)
sp500 <- ts(sp500b$sp500, start=c(1), end=c(1258))

reg22 <- arima(sp500, order = c(1,1,1))
summary(reg22)

arima(sp500, order = c(2,1,2))
arima(sp500, order = c(3,1,3))

#Table 10.9;
install.packages("forecast")
library(forecast)

sp500b = read.csv("sp500_oil_2010_2014.csv", header = TRUE)
sp500 <- ts(sp500b$sp500)
reg23 <-arima(sp500, order = c(1,1,1))
reg24 <-arima(sp500, order = c(2,1,2))
reg25 <-arima(sp500, order = c(3,1,3))

# The following command produces the forecasts for the next 5
trading days
#  for the S&P500. This is different from Table 10.9 in the
book, which
#  uses the updated daily values of the S&P500 for the
predictions into
#  the first 5 trading days of 2015.  The forecasts can be
compared
#  to actual values in Table 10.9.  The "forecast" command also
gives
#  prediction intervals.
forecast(reg23, h=5)
```

Section 10.9

```
#GARCH model;
```

```
# Load required packages
library(readr)      # for import delimited
library(tseries)    # for adf.test
library(rugarch)    # for GARCH and EGARCH models

# --- 1. Import data ---
data <- read_csv("C:/data/Regression-
textbook/data/datasets/SP500.csv")

# Assume variable sp500_pct exists as percent returns
sp500 <- data$sp500_pct
t <- seq_along(sp500)  # equivalent to g t = _n

# --- 2. Augmented Dickey-Fuller test (equiv. to dfuller) ---
adf_result <- adf.test(sp500)
print(adf_result)

# --- 3. Standard GARCH(1,1) model ---
spec_garch <- ugarchspec(
  variance.model = list(model = "sGARCH", garchOrder = c(1, 1)),
  mean.model     = list(armaOrder = c(0, 0), include.mean =
TRUE),
  distribution.model = "norm"
)
fit_garch <- ugarchfit(spec = spec_garch, data = sp500)
show(fit_garch)

# --- 4. EGARCH(1,1) model (Nelson exponential GARCH) ---
spec_egarch <- ugarchspec(
  variance.model = list(model = "eGARCH", garchOrder = c(1, 1)),
  mean.model     = list(armaOrder = c(0, 0), include.mean =
TRUE),
  distribution.model = "norm"
)
fit_egarch <- ugarchfit(spec = spec_egarch, data = sp500)
show(fit_egarch)
```